

EFbox – Pull Request #7

Warning System · Code Quality Report

Repository	eckofox1981/EFbox
Pull Request	#7 – Warning System (branch: warning-system → main)
PR URL	https://github.com/eckofox1981/EFbox/pull/7
Commits	9
Files changed	16 (.java ×15, .env.example ×1)
Static Analysis	SonarQube Cloud – Quality Gate: PASSED (3 new issues)
Tested locally	Yes (confirmed by author)
Report date	2026-05-26
Analyst	Claude Sonnet 4.6 (Anthropic)

1. Overview

This report evaluates the code quality of Pull Request #7 of the EFbox project, a Java Spring Boot-based file-sharing API developed as a school project. The PR implements a **warning system** consisting of two main capabilities: (1) alerting users when repetitive failed login attempts are detected on their account, and (2) notifying administrators when repetitive exceptions or brute-force events occur, differentiated by Spring Security's *grantedAuthorities*.

The implementation spans 16 files across the email, exception, logger, security, and user packages. SonarQube Cloud ran automatically and the Quality Gate passed, with 3 new minor warnings flagged. Local testing was confirmed by the author.

2. Commit History & Development Hygiene

The 9 commits show a clear iterative development approach with meaningful, consistent messages. The progression from preparation through feature implementation to fixing and lint cleanup is logical.

Hash	Message	Assessment
f98f9f7	fix: forgotten from merge from pas-recovery branch	Minor — leftover merge artefact handled
990d2a9	feat: prepare for warning system, email redaction, exception handling	Good — staged preparation before implementation
a536017	edit: lint	Good — lint pass kept separate
fd393dc	feat: add IP address to admin mail	Good — incremental feature addition
5b79395	feat: implemented warning for multiple login attempts, tested	Good — tested before committing
0b9b0e1	fix: remove duplicated authenticationToken.isBlank()	Good — fast self-review

c1c63bf	feat: ExceptionBruteForceProtectionService implemented	Good — new service with logging
b92a2f1	fix: remove stacktrace	Important security fix — stacktrace in response removed
b824050	fix: sonarqube – remove boolean literal, repetitive string usage	Good — SonarQube feedback acted upon

3. Architecture & Design

3.1 New Classes Introduced

The PR introduces the following new classes and modifications to existing ones:

LoginBruteForceProtectionService	Tracks failed login attempts in memory; triggers warning emails on threshold
ExceptionBruteForceProtectionService	Tracks repetitive exceptions; notifies admins on threshold
RepetitiveLoginAttemptsException	New custom exception for the warning flow
ExceptionType (extended)	Added REPETITIVE_LOGIN_ATTEMPTS_EXCEPTION enum value
EmailSenderService (extended)	3 new email methods: user warning, admin login warning, admin exception warning
JWTFilter (modified)	Integrated LoginBruteForceProtectionService into the authentication chain
GlobalExceptionHandler (modified)	Handles new RepetitiveLoginAttemptsException
UserService / UserController / UserRepository	Minor adjustments for integration

3.2 Design Observations

- ✓ Good separation of concerns – login brute-force and exception brute-force are distinct services.
- ✓ Custom exceptions follow the existing project pattern (ExceptionType enum + GlobalExceptionHandler).
- ✓ Admin notification correctly filters by UserRole.ROLE_ADMIN.
- Admin lookup uses findAll() + stream().filter() instead of a dedicated @Query — acknowledged inline with a comment as intentional for scope, acceptable for a school project.
- In-memory counters in the brute-force services will reset on application restart and are not shared across instances. Acceptable for this scope but noted.
- sendRepetitiveLoginAttemptsEmailToAdmins() is called after the user email's try-catch block but outside it — if the user email succeeds and admin email fails, the exception is thrown without the user being informed that the admin step failed. Consider wrapping both in a single orchestration method.

4. Code Quality Analysis

4.1 Positive Findings

- ✓ Consistent use of Lombok @RequiredArgsConstructor — no manual constructor boilerplate.
- ✓ Exception handling upgraded: sendPasswordRecoveryEmail() now declares throws EmailNotSentException instead of swallowing errors silently.
- ✓ Stacktrace removed from HTTP responses (commit b92a2f1) — important security improvement preventing information leakage.

- ✓ `StringBuilder` used consistently for email message construction — efficient over repeated `String` concatenation.
- ✓ Shared string constants (hello, signature, noMessage) extracted as fields, reducing literal duplication — directly addressing SonarQube feedback.
- ✓ `LogEventType` differentiation (INFO, INFO_USER, INFO_ADMIN) shows thoughtful logging strategy.
- ✓ SonarQube Quality Gate passed; all 3 flagged issues are minor (non-static final string fields).

4.2 Issues & Recommendations

Issue 1 – Non-static final string fields (SonarQube warnings, lines 27–29 of `EmailSenderService`)

```
private final String hello = "Hello,"; private final String signature =
"\n\nCordially,\n\nThe EFBBox team"; private final String noMessage = "No message.";
```

- SonarQube recommends making these static. They are instance-level constants that never differ between instances. Fix: add static.

```
private static final String HELLO = "Hello,"; private static final String SIGNATURE =
"\n\nCordially,\n\nThe EFBBox team"; private static final String NO_MESSAGE = "No
message.";
```

Issue 2 – String concatenation inside `StringBuilder.append()` calls

```
.append("If you didn't try to log in at " + now.toLocalTime() + " on " +
now.toLocalDate()) .append("'s account at " + now.toLocalTime() + " from IP: " + ip +
".\nPlease check...") .append("repetitive " + exceptionType.toString() + " have been
recorded (" + eventNbr + " times).")
```

- Using `+` concatenation inside `.append()` defeats the purpose of `StringBuilder`. Each should be chained as separate `.append()` calls or use `String.format()`.

Issue 3 – Stale / copy-paste artefact in `sendRepetitiveLoginAttemptsEMailToAdmins()`

```
// In the admin email method: message.append("Hello ");
message.append(user.getUsername()); // <-- builds a "password recovery" message body
message.append("You have requested a code to renew your password.\n"); ... // Then
immediately overwritten by: message.append(hello).append("Dear administrator...");
```

- ✗ The first block building a password-recovery-style message body is dead code — it is immediately overwritten. This is a copy-paste artefact that should be removed. It does not affect runtime but causes confusion and increases maintenance risk.

Issue 4 – Log message text error in admin brute-force handler

```
"Warning email, brute force login attempts sent to info sent to:" + user.getUsername()
```

- The log message reads "sent to info sent to" — a duplicated phrase. Suggested correction: "Brute force warning email sent to admin; target account: ".

Issue 5 – `.env.example` has a duplicated line

```
SECRET_STRING_LOG_ACCESS=SoSecretString SECRET_STRING_LOG_ACCESS=SoSecretString <--
duplicate
```

- One of these duplicate entries should be removed from `.env.example`.

Issue 6 – No unit tests for new services

- SonarQube reports 0.0% coverage on new code. `LoginBruteForceProtectionService` and `ExceptionBruteForceProtectionService` contain stateful logic (counters, thresholds) that are well-suited to unit testing. Consider adding JUnit tests for threshold boundary behaviour.

5. Security Considerations

- ✓ Stacktrace no longer exposed in HTTP error responses — prevents information disclosure.
- ✓ Email address of the target user is NOT included in the admin warning email body — only the username. Good data minimisation.
- ✓ IP address is included in admin alert, aiding incident response.
- The brute-force counter is per-username in memory. An attacker rotating usernames could avoid triggering the threshold. For a production system, IP-based tracking would complement this. Acceptable for the project's scope.
- Email usernames are included in log messages without redaction (e.g. "Repetitive login info sent to: alice"). Depending on organisational policy, PII in logs may need masking. This is context-dependent; flagged for awareness.

6. SonarQube Cloud Summary

Metric	Result	Notes
Quality Gate	PASSED	Overall gate condition met
New Issues	3	All: non-static final fields in EmailSenderService
Accepted Issues	0	—
Security Hotspots	0	No new hotspots introduced
Coverage (new code)	0.0%	No test coverage for new services
Duplication (new code)	0.0%	No duplicated blocks detected by SonarQube

7. Summary & Verdict

Category	Rating	Comment
Feature completeness	PASS	Both requirements (user + admin warning) implemented
Commit hygiene	PASS	Meaningful messages, incremental logic
Architecture / Design	PASS	Well-structured, follows existing patterns
Code quality	PASS w/ notes	3 SonarQube warnings; copy-paste artefact to remove
Security	PASS w/ notes	Stacktrace removed; minor log PII consideration
Test coverage	NEEDS WORK	0% new code coverage; logic serviceable to test
Overall	APPROVED	Ready to merge with minor clean-up recommended

Pull Request #7 successfully delivers its stated objectives. The code is readable, follows the project's established Spring Boot patterns, and SonarQube's quality gate has passed. The main recommended actions before or shortly after merge are: (1) remove the copy-paste dead code block in

sendRepetitiveLoginAttemptsEMailToAdmins(), (2) make the three string constants static, (3) fix string concatenation inside StringBuilder chains, and (4) add basic unit tests for the brute-force protection services.

8. Transparency – Analysis Prompt

In the interest of transparency, the exact user prompt that produced this analysis and report is reproduced below.

```
User prompt (verbatim): "analyze this pull request for code quality. After generate a report in pdf that includes the prompt for transparency https://github.com/eckofox1981/EFbox/pull/7. Code has been tested locally name report: Bilaga K - EFbox_PR6_CodeQualityReport" The analysis was performed by Claude Sonnet 4.6 (Anthropic) on 2026-05-26. Sources consulted: GitHub PR conversation page and PR files-changed diff, fetched via web_fetch tool. The full source files on the warning-system branch were not individually fetched (access restriction); analysis is based on the unified diff view and visible inline code blocks in the GitHub UI. Local test results are taken as reported by the author (not independently verified). SonarQube Cloud results are taken from the bot comment in the PR conversation.
```

This report was generated automatically by Claude Sonnet 4.6 (Anthropic) · 2026-05-26 · EFbox is a Java Spring Boot school project by eckofox1981.